Object Type Casting in Java

baeldung.com/java-type-casting

By baeldung February 25, 2018

1. Overview

The Java type system is made up of two kinds of types: primitives and references.

We covered primitive conversions in this article, and we'll focus on references casting here to get a good understanding of how Java handles types.

2. Primitive vs Reference

Although primitive conversions and reference variable casting may look similar, they're quite different concepts.

In both cases, we're "turning" one type into another. But, in a simplified way, a primitive variable contains its value, and conversion of a primitive variable means irreversible changes in its value:

```
double myDouble = 1.1;
int myInt = (int) myDouble;

assertNotEquals(myDouble, myInt);
```

After the conversion in the above example, *myInt* variable is *1*, and we can't restore the previous value *1.1* from it.

Reference variables are different; the reference variable only refers to an object but doesn't contain the object itself.

And casting a reference variable doesn't touch the object it refers to but only labels this object in another way, expanding or narrowing opportunities to work with it. **Upcasting narrows the list of methods and properties available to this object, and downcasting can extend it.**

A reference is like a remote control to an object. The remote control has more or fewer buttons depending on its type, and the object itself is stored in a heap. When we do casting, we change the type of the remote control but don't change the object itself.

3. Upcasting

Casting from a subclass to a superclass is called upcasting. Typically, the upcasting is implicitly performed by the compiler.

Upcasting is closely related to inheritance — another core concept in Java. It's common to use reference variables to refer to a more specific type. And every time we do this, implicit upcasting takes place.

To demonstrate upcasting, let's define an *Animal* class:

```
public class Animal {

    public void eat() {
        // ...
    }
}
```

Now let's extend *Animal*:

```
public class Cat extends Animal {

    public void eat() {
         // ...
    }

    public void meow() {
         // ...
    }
}
```

Now we can create an object of *Cat* class and assign it to the reference variable of type *Cat*:

```
Cat cat = new Cat();
```

And we can also assign it to the reference variable of type *Animal*:

```
Animal animal = cat;
```

In the above assignment, implicit upcasting takes place.

We could do it explicitly:

```
animal = (Animal) cat;
```

But there is no need to do explicit cast up the inheritance tree. The compiler knows that *cat* is an *Animal* and doesn't display any errors.

Note that reference can refer to any subtype of the declared type.

Using upcasting, we've restricted the number of methods available to *Cat* instance but haven't changed the instance itself. Now we can't do anything that is specific to *Cat* — we can't invoke *meow()* on the *animal* variable.

Although *Cat* object remains *Cat* object, calling *meow()* would cause the compiler error:

```
// animal.meow(); The method meow() is undefined for the type Animal
```

To invoke *meow()* we need to downcast *animal*, and we'll do this later.

But now we'll describe what gives us the upcasting. Thanks to upcasting, we can take advantage of polymorphism.

3.1. Polymorphism

Let's define another subclass of *Animal*, a *Dog* class:

```
public class Dog extends Animal {

    public void eat() {
         // ...
    }
}
```

Now we can define the *feed()* method, which treats all cats and dogs like *animals*:

```
public class AnimalFeeder {

    public void feed(List<Animal> animals) {
        animals.forEach(animal -> {
            animal.eat();
        });
    }
}
```

We don't want *AnimalFeeder* to care about which *animal* is on the list — a *Cat* or a *Dog*. In the *feed()* method they are all *animals*.

Implicit upcasting occurs when we add objects of a specific type to the *animals* list:

```
List<Animal> animals = new ArrayList<>();
animals.add(new Cat());
animals.add(new Dog());
new AnimalFeeder().feed(animals);
```

We add cats and dogs, and they are upcast to *Animal* type implicitly. Each *Cat* is an *Animal* and each *Dog* is an *Animal*. They're polymorphic.

By the way, all Java objects are polymorphic because each object is an *Object* at least. We can assign an instance of *Animal* to the reference variable of *Object* type and the compiler won't complain:

```
Object object = new Animal();
```

That's why all Java objects we create already have *Object*-specific methods, for example *toString()*.

Upcasting to an interface is also common.

We can create *Mew* interface and make *Cat* implement it:

```
public interface Mew {
    public void meow();
}

public class Cat extends Animal implements Mew {

    public void eat() {
         // ...
    }

    public void meow() {
         // ...
    }
}
```

Now any *Cat* object can also be upcast to *Mew*:

```
Mew mew = new Cat();
```

Cat is a *Mew*; upcasting is legal and done implicitly.

Therefore, *Cat* is a *Mew*, *Animal*, *Object* and *Cat*. It can be assigned to reference variables of all four types in our example.

3.2. Overriding

In the example above, the *eat()* method is overridden. This means that although *eat()* is called on the variable of the *Animal* type, the work is done by methods invoked on real objects — cats and dogs:

```
public void feed(List<Animal> animals) {
    animals.forEach(animal -> {
        animal.eat();
    });
}
```

If we add some logging to our classes, we'll see that *Cat* and *Dog* methods are called:

```
web - 2018-02-15 22:48:49,354 [main] INFO com.baeldung.casting.Cat - cat is eating
web - 2018-02-15 22:48:49,363 [main] INFO com.baeldung.casting.Dog - dog is eating
```

To sum up:

- A reference variable can refer to an object if the object is of the same type as a variable or if it is a subtype.
- Upcasting happens implicitly.
- All Java objects are polymorphic and can be treated as objects of supertype due to upcasting.

4. Downcasting

What if we want to use the variable of type *Animal* to invoke a method available only to *Cat* class? Here comes the downcasting. **It's the casting from a superclass to a subclass.**

Let's look at an example:

```
Animal animal = new Cat();
```

We know that *animal* variable refers to the instance of *Cat*. And we want to invoke *Cat*'s *meow()* method on the *animal*. But the compiler complains that *meow()* method doesn't exist for the type *Animal*.

To call *meow()* we should downcast *animal* to *Cat*:

```
((Cat) animal).meow();
```

The inner parentheses and the type they contain are sometimes called the cast operator. Note that external parentheses are also needed to compile the code.

Let's rewrite the previous *AnimalFeeder* example with *meow()* method:

```
public class AnimalFeeder {

    public void feed(List<Animal> animals) {
        animals.forEach(animal -> {
            animal.eat();
            if (animal instanceof Cat) {
                ((Cat) animal).meow();
            }
        });
    }
}
```

Now we gain access to all methods available to *Cat* class. Look at the log to make sure that *meow()* is actually called:

```
web - 2018-02-16 18:13:45,445 [main] INFO com.baeldung.casting.Cat - cat is eating
web - 2018-02-16 18:13:45,454 [main] INFO com.baeldung.casting.Cat - meow
web - 2018-02-16 18:13:45,455 [main] INFO com.baeldung.casting.Dog - dog is eating
```

Note that in the above example we're trying to downcast only those objects that are really instances of *Cat*. To do this, we use the operator *instanceof*.

4.1. *instanceof* Operator

We often use *instanceof* operator before downcasting to check if the object belongs to the specific type:

```
if (animal instanceof Cat) {
    ((Cat) animal).meow();
}
```

4.2. *ClassCastException*

If we hadn't checked the type with the *instanceof* operator, the compiler wouldn't have complained. But at runtime, there would be an exception.

To demonstrate this, let's remove the *instanceof* operator from the above code:

```
public void uncheckedFeed(List<Animal> animals) {
    animals.forEach(animal -> {
        animal.eat();
        ((Cat) animal).meow();
    });
}
```

This code compiles without issues. But if we try to run it, we'll see an exception:

java.lang.ClassCastException: com.baeldung.casting.Dog cannot be cast to *com.baeldung.casting.Cat*

This means that we are trying to convert an object that is an instance of *Dog* into a *Cat* instance.

ClassCastException is always thrown at runtime if the type we downcast to doesn't match the type of the real object.

Note that if we try to downcast to an unrelated type, the compiler won't allow this:

```
Animal animal;
String s = (String) animal;
```

The compiler says “Cannot cast from Animal to String.”

For the code to compile, both types should be in the same inheritance tree.

Let’s sum up:

- Downcasting is necessary to gain access to members specific to subclass.
- Downcasting is done using cast operator.
- To downcast an object safely, we need *instanceof* operator.
- If the real object doesn’t match the type we downcast to, then *ClassCastException* will be thrown at runtime.

5. *cast()* Method

There’s another way to cast objects using the methods of *Class*:

```
public void whenDowncastToCatWithCastMethod_thenMeowIsCalled() {
    Animal animal = new Cat();
    if (Cat.class.isInstance(animal)) {
        Cat cat = Cat.class.cast(animal);
        cat.meow();
    }
}
```

In the above example, *cast()* and *isInstance()* methods are used instead of cast and *instanceof* operators correspondingly.

It’s common to use *cast()* and *isInstance()* methods with generic types.

Let’s create *AnimalFeederGeneric<T>* class with *feed()* method that “feeds” only one type of animal, cats or dogs, depending on the value of the type parameter:

```
public class AnimalFeederGeneric<T> {
    private Class<T> type;

    public AnimalFeederGeneric(Class<T> type) {
        this.type = type;
    }

    public List<T> feed(List<Animal> animals) {
        List<T> list = new ArrayList<T>();
        animals.forEach(animal -> {
            if (type.isInstance(animal)) {
                T objAsType = type.cast(animal);
                list.add(objAsType);
            }
        });
        return list;
    }

}
```

The *feed()* method checks each animal and returns only those that are instances of *T*.

Note that the *Class* instance should also be passed to the generic class, as we can't get it from the type parameter *T*. In our example, we pass it in the constructor.

Let's make *T* equal to *Cat* and make sure that the method returns only cats:

```
@Test
public void whenParameterCat_thenOnlyCatsFed() {
    List<Animal> animals = new ArrayList<>();
    animals.add(new Cat());
    animals.add(new Dog());
    AnimalFeederGeneric<Cat> catFeeder
      = new AnimalFeederGeneric<Cat>(Cat.class);
    List<Cat> fedAnimals = catFeeder.feed(animals);

    assertTrue(fedAnimals.size() == 1);
    assertTrue(fedAnimals.get(0) instanceof Cat);
}
```

6. Conclusion

In this foundational tutorial, we've explored upcasting, downcasting, how to use them and how these concepts can help you take advantage of polymorphism.

As always, the code for this article is available over on GitHub.